

Database Development Projektarbeit

Du entwickelst eine kleine, lauffähige Anwendung mit Spring Boot und PostgreSQL. Dabei zeigst du, dass du Datenmodell, Anwendung, Datenzugriffe, Konsistenz und Performance als zusammenhängende Aufgabe bearbeiten kannst. Du begründest deine Entscheidungen und belegst sie mit Tests und Messungen.

Rahmen und Ziel

Du arbeitest **allein**. Zur Verfügung stehen **sechs Projektlektionen inklusive Einführung und 2–3 Stunden Hausarbeit**. Zusätzlich werden drei Lektionen zum Thema Performance durchgeführt. Du teilst deine Arbeit selbst ein; es gibt keine vorgeschriebenen Arbeitsphasen oder Zwischenmeilensteine. In einer zusätzlichen Schlusslektion verteidigst du dein Projekt während **5–10 Minuten vor der Klasse** (maximal 5 Minuten Präsentation und maximal 5 Minuten Rückfragen). Die Durchführung organisiert der Dozent.

Wähle ein eigenes Thema und definiere **fünf bis sieben überprüfbare fachliche Anforderungen**. Alle vereinbarten fachlichen Anforderungen und alle technischen Anforderungen dieses Blatts sind Pflicht. Zusätzliche Funktionen ersetzen keine fehlenden Pflichtanforderungen.

Halte den fachlichen Umfang klein. Eine API, die du beispielsweise über Swagger UI oder HTTP-Anfragen demonstrieren kannst, genügt. Ein Frontend, Benutzeranmeldung und externe Dienste sind nicht erforderlich. Das im Unterricht behandelte Ticketsystem ist keine Projektvariante. Technische Grundstrukturen aus dem Kurs dürfen mit Herkunftsangabe wiederverwendet werden; deine fachliche Umsetzung und deine Entscheidungen müssen eigenständig nachvollziehbar sein.

Projektsteckbrief: Abgabe über Moodle

Reiche deinen Projektsteckbrief als PDF über Moodle ein. Die Abgabetermine werden dort bekanntgegeben. Der Steckbrief enthält:

Bestandteil	Erwarteter Inhalt
Thema	Name, Ausgangslage und Nutzen deiner Anwendung
Anforderungen	Fünf bis sieben nummerierte, fachliche Anforderungen mit überprüfbarem Ergebnis
Modellskizze	Erste fachliche Objekte, Beziehungen und Kardinalitäten; von dir selbst entworfen
Kritische Geschäftsoperation	Welche zusammengehörende Änderung muss vollständig gelingen oder scheitern? Welcher Konflikt kann bei gleichzeitiger Bearbeitung auftreten?
Vorgesehene Auswertungen	Welche fachlichen Fragen soll die Anwendung mit ihren Daten beantworten?

Formuliere Anforderungen aus Sicht der Nutzung: Welches Ergebnis wird erwartet und welche Geschäftsregel muss gelten? Kläre den Umfang mit dem Dozenten. Tabellen, Endpunkte, Abfragen, Transaktionsgrenzen und Lösungswege werden nicht vorgegeben.

Themenideen

Diese Ideen dienen als Anregung. Die genannten Funktionen sind mögliche Anforderungen, keine zusätzliche Pflichtliste. Wähle einen überschaubaren Umfang und formuliere daraus deinen eigenen vollständigen Auftrag. Eigene Themen sind ebenfalls möglich, sofern sie alle technischen Anforderungen ermöglichen.

Ferienwohnungen

Eine kleine Vermietung möchte Wohnungen und Buchungen verwalten. Mögliche Anforderungen: Wohnungen erfassen und deaktivieren; Gäste verwalten; Aufenthalte buchen und stornieren; Doppelbuchungen verhindern; freie Wohnungen für einen Zeitraum suchen; Buchungshäufigkeit oder Umsatz auswerten.

Abgrenzung: Keine Zahlungsabwicklung und keine Anbindung an Buchungsportale. Lege fest, welche Buchungen in eine Auswertung eingehen.

Kinoreservationen

Ein Kino möchte Vorstellungen und Reservationen verwalten. Mögliche Anforderungen: Filme und Vorstellungen erfassen; Plätze reservieren und Reservationen stornieren; doppelte Sitzplatzvergabe verhindern; verfügbare Plätze anzeigen; Reservationen einer Person suchen; Auslastung je Film auswerten.

Abgrenzung: Kein grafischer Sitzplan und kein Bezahlssystem.

Carsharing

Ein Anbieter möchte Fahrzeuge und Reservationen verwalten. Mögliche Anforderungen: Fahrzeuge erfassen und ausser Betrieb nehmen; Kunden verwalten; Fahrzeuge reservieren und Reservationen stornieren; Überschneidungen verhindern; verfügbare Fahrzeuge suchen; Nutzungsdauer je Fahrzeug auswerten.

Abgrenzung: Keine Fahrzeugortung und keine Tarifautomatik.

Restaurantreservationen

Ein Restaurant möchte Tische und Reservationen verwalten. Mögliche Anforderungen: Tische mit Kapazität erfassen; Reservationen anlegen, ändern und stornieren; Überbelegungen verhindern; verfügbare Tische suchen; Tagesreservationen anzeigen; Gruppengröße oder Auslastung auswerten.

Abgrenzung: Ein Restaurant genügt. Keine automatische Kombination mehrerer Tische; definiere eindeutige Reservationszeiträume.

Pakettracking

Eine Versandstelle möchte Sendungen und ihren Verlauf nachvollziehen. Mögliche Anforderungen: Sendungen erfassen; Statusänderungen mit Verlauf aufzeichnen; unzulässige Übergänge verhindern; aktuellen Status anzeigen; nach Kunde, Status und Zeitraum suchen; lange unveränderte Sendungen und Statusverteilungen auswerten.

Abgrenzung: Keine Schnittstellen zu Versanddienstleistern und keine Live-Ortung.

Fitness- und Kursbuchungen

Ein Fitnesscenter möchte Mitglieder und Kursteilnahmen verwalten. Mögliche Anforderungen: Mitglieder und Kurse erfassen; Anmeldungen und Abmeldungen ermöglichen; doppelte Anmeldungen und Überbuchungen verhindern; freie Plätze anzeigen; Teilnahmen eines Mitglieds suchen; Auslastung je Kurs auswerten.

Abgrenzung: Keine Abonnements und keine wiederkehrende Terminplanung.

Technische Pflichtenforderungen

Die Kennungen T1–T12 dienen der Zuordnung deiner Nachweise. Eine Funktion darf mehrere Anforderungen erfüllen, sofern du jeden geforderten Aspekt nachweist.

T1 – Datenmodell und Integrität

Entwirf mindestens drei fachlich zusammenhängende Tabellen. Dokumentiere dein Modell mit einem ER-Diagramm samt Kardinalitäten, Primär- und Fremdschlüsseln. Wähle geeignete Datentypen und sinnvolle Constraints für Pflichtwerte, Eindeutigkeit und gültige Werte. Begründe Normalisierung und allfällige bewusste Redundanz. Zentrale Datenbankregeln dürfen nicht ausschliesslich von API-Validierung abhängen.

Nachweis: ER-Diagramm, Migrationen und kurze Begründung deiner Schemaentscheidungen.

T2 – Migrationen

Baue die Datenbank reproduzierbar mit Flyway auf. Setze mindestens eine nachfolgende Schemaänderung als neue Migration um. Bereits angewendete Migrationen bleiben unverändert. Die Anwendung muss mit einer leeren Datenbank eingerichtet werden können; auch der Übergang vom früheren zum erweiterten Schema muss funktionieren.

Nachweis: Versionierte Migrationen und nachvollziehbare Prüfung von Neuaufbau und Schemaentwicklung.

T3 – Anwendung und JPA

Implementiere deine fachlichen Anforderungen mit Spring Boot und PostgreSQL. Nutze JPA für Persistenz und Beziehungen. Stelle Erfassung, Lesen, Änderung sowie fachlich sinnvolles Löschen oder Deaktivieren bereit. Trenne API-DTOs von Entities und Geschäftslogik vom Datenzugriff. Nicht jede Tabelle benötigt eine vollständige CRUD-API.

Nachweis: Lauffähige API und reproduzierbare Beispielaufufe für deine fachlichen Anforderungen.

T4 – Transaktion und Rollback

Implementiere eine fachliche Operation, die mehrere zusammengehörende Schreibzugriffe atomar ausführt. Bei einem Fehler dürfen keine Teiländerungen bestehen bleiben. Eine reine Eingabepfung vor dem ersten Schreibzugriff belegt noch keinen Rollback.

Nachweis: Erfolgsfall und automatisierter Fehlerfall, bei dem ein Fehler nach einem bereits erfolgten Schreibzugriff die vollständige Rücknahme belegt.

T5 – Gleichzeitige Änderungen

Verhindere, dass konkurrierende Änderungen eine wichtige Geschäftsregel verletzen oder Änderungen unbemerkt überschreiben. Leite die Konfliktbehandlung aus deiner Domäne ab. Die Strategie wählst und begründest du selbst.

Nachweis: Reproduzierbarer Test mit zwei unabhängigen, konkurrierenden Zugriffen, geprüftem Endzustand und nachvollziehbarer Konfliktrückmeldung. Zwei gewöhnliche Aufrufe nacheinander genügen nicht.

T6 – Query Design, SQL/JDBC und View

Nutze eine gezielte JPQL-/DTO-Projektion für einen Leseanwendungsfall. Implementiere zusätzlich eine parametrisierte SQL-Auswertung über JDBC mit JOIN und Aggregation. Erstelle und verwende eine Datenbank-View; sie darf Teil derselben Auswertung sein. Die Auswertung muss auch bei mehreren zugehörigen Datensätzen korrekt zählen beziehungsweise summieren.

Nachweis: Verwendete Abfragen und View, API-Aufruf, begründete Wahl der Zugriffsformen sowie Test mit fachlich bekannten Ergebnissen.

Datenzugriffe prüfen und verbessern

T7 – Filter, Sortierung und Pagination

Eine relevante Liste muss datenbankseitig gefiltert, stabil sortiert und seitenweise geladen werden. Vermeide das vollständige Laden aller Datensätze mit anschliessender Filterung oder Seiteneinteilung im Anwendungsspeicher.

Nachweis: Aufrufe verschiedener Seiten und Filter, Prüfung der Reihenfolge und ein Nachweis der begrenzten Datenbankabfrage.

T8 – Integrationstests mit PostgreSQL

Prüfe die wesentlichen Geschäftsregeln, Transaktionen, Konflikte und Abfrageergebnisse automatisiert gegen echtes PostgreSQL, beispielsweise mit Testcontainers. Mocks oder eine andere Datenbank allein genügen nicht. Verwende kleine, kontrollierte Testdaten für fachliche Tests; der grosse Datenbestand aus T9 muss nicht bei jedem Test aufgebaut werden.

Nachweis: Ausführbare Tests für gültige und ungültige Eingaben beziehungsweise Zustände, Rollback, konkurrierende Änderungen und korrekte Abfrageergebnisse. Dokumentiere den Testaufruf und seine Voraussetzungen.

T9 – Reproduzierbare Testdaten

Erzeuge mindestens **100'000 Datensätze in einer relevanten Vorgangstabelle** sowie passende zugehörige Daten. Wähle eine Verteilung, mit der deine Filter und Auswertungen sinnvoll untersucht werden können. Die Erzeugung muss wiederholbar sein und darf keine echten Personendaten verwenden.

Nachweis: Skript oder Generator, Aufruf, Datensatzanzahlen und kurze Beschreibung der Datenverteilung.

T10 – Performanceanalyse und Index

Wähle eine relevante Abfrage. Dokumentiere SQL, Parameter, Datenmenge und Ausgangszustand. Analysiere sie mit EXPLAIN ANALYZE, führe eine begründete Optimierung durch und vergleiche die Ergebnisse unter gleichen Bedingungen. Implementiere mindestens einen fachlich begründeten Index und erläutere seinen Nutzen sowie seine Kosten.

Nachweis: Ausführungspläne und wiederholte Messungen vor und nach der Änderung, Beschreibung der Umgebung und Erklärung des Ergebnisses. Trenne die Untersuchung der Datenbankabfrage vom Anwendungscache. Ein bestimmter Beschleunigungsfaktor ist nicht vorgeschrieben; eine ausbleibende Verbesserung muss analysiert und die Entscheidung begründet werden. Der Ausgangszustand muss reproduzierbar bleiben.

T11 – ORM-Ladeverhalten

Untersuche einen Lesezugriff mit Beziehungen auf N+1 und unnötige Datenladung. Begründe deine Ladeentscheidung anhand des tatsächlichen Datenbedarfs. Verbessere ein gefundenes Problem; wenn keines vorliegt, belege dies.

Nachweis: SQL-Log oder Abfragezählung für unterschiedlich grosse Ergebnislisten, Erklärung der geladenen Daten und gegebenenfalls Vorher-/Nachher-Vergleich.

T12 – Cache und Aktualität

Setze einen Cache für einen sinnvollen Leseanwendungsfall ein. Definiere, welche Ergebnisse unter welchen Schlüsseln gespeichert werden und wie Änderungen berücksichtigt werden. Begründe die Strategie aus Sicht der fachlich erforderlichen Aktualität.

Nachweis: Erster Zugriff, wiederholter Zugriff mit Cache-Treffer und Zugriff nach einer relevanten Datenänderung. Belege, dass die definierte Aktualitätsregel eingehalten wird; eine Cache-Annotation allein genügt nicht.

Projektabgabe über GitHub

Gib dein Projekt über ein GitHub-Repository ab. Lade den Dozenten unter **simon.erhardt@hftm.ch** zum Zugriff ein und stelle sicher, dass er das Repository zur Bewertung öffnen kann. Der verbindliche Abgabetermin wird in Moodle bekanntgegeben. Kennzeichne den abgegebenen Stand im README mit einem eindeutigen Git-Tag.

Das Repository enthält:

- Quellcode, Build-Konfiguration und alle Flyway-Migrationen;
- Testdatengenerator beziehungsweise Skripte und automatisierte Tests;
- reproduzierbare API-Beispielaufufe;
- Performance-Nachweise und Messungen;
- ein kompaktes README und die erforderlichen Erklärungen zum Einsatz von Hilfsmitteln.

Inhalt des README

Beschreibe Thema und fachliche Anforderungen. Zeige das ER-Diagramm und begründe zentrale Entscheidungen zu Datenmodell, Transaktion, Nebenläufigkeit, Abfragen, Ladeverhalten und Cache. Dokumentiere Voraussetzungen und konkrete Aufrufe für Start, Datenbankaufbau, Testdatenerzeugung und Tests. Verwende keine persönlichen Zugangsdaten oder Geheimnisse im Repository.

Ordne jede fachliche Anforderung und T1–T12 einem Nachweis zu. Nutze beispielsweise folgende Struktur; die Einträge füllst du mit deinen eigenen Ergebnissen:

Anforderung	Umsetzung	Nachweis
Kennung und Kurzname	Relevante Datei oder Funktion	Test, API-Aufruf, Messung oder Dokumentation

Ein Verweis auf ausführbare Nachweise ist aussagekräftiger als viele Screenshots. Benenne verbleibende Einschränkungen ehrlich. Das README darf auf ergänzende Dateien im Repository verlinken.

KI und weitere Hilfsmittel

Es gelten die **HFTM-KI-Richtlinien für Studierende**, Version 1.1 vom 02.02.2026: [KI an der hftm einsetzen \(Studierende\)](#).

Für die Projektarbeit ist KI-Unterstützung im Rahmen dieser Richtlinie zulässig. Du steuerst die Arbeit, prüfst Ergebnisse kritisch und trägst die Verantwortung für deine Abgabe. Unreflektierte Übernahme ist nicht zulässig. Vertrauliche oder personenbezogene Daten dürfen nicht in KI-Tools eingegeben werden.

Lege deiner Abgabe bei:

- **Eigenständigkeitserklärung** einschliesslich der Erklärung deines KI-Einsatzes;
- **Hilfsmittelverzeichnis** mit Werkzeug, Einsatzzweck, betroffenen Stellen beziehungsweise Dateien sowie Version/Datum;
- **Prompt-Verzeichnis** mit Ziel, Prompt, Nutzung des Outputs, eigener Bearbeitung und betroffener Stelle;
- nachvollziehbare Kennzeichnung der KI-Unterstützung an den betroffenen Stellen oder im Arbeitsjournal sowie Herkunftsangaben für übernommenes Material.

Wenn du keine KI eingesetzt hast, halte dies ausdrücklich fest. KI-Ausgaben ersetzen keine fachlichen Quellen. Massgeblich bleibt die verlinkte Richtlinie.

Verteidigung

Die Verteidigung dauert insgesamt **5–10 Minuten**. Für deine **Präsentation stehen maximal 5 Minuten** zur Verfügung; zeige eine kurze Demonstration und begründe eine wichtige Datenbankentscheidung. Anschliessend sind **maximal 5 Minuten für Rückfragen** vorgesehen. Zeige die Anwendung vorbereitet und erläutere die relevanten Nachweise. Du musst deine Lösung selbst erklären können. Die Verteidigung ist eine individuelle Leistung ohne KI-Unterstützung während der Befragung.

Bewertungsraster

Bewertet werden die abgegebene Umsetzung, ihre Nachweise und dein Verständnis in der Verteidigung. Alle Pflichtanforderungen bleiben verbindlich. Fehlende oder mangelhafte Teile führen zu Punktabzug im jeweiligen Teilkriterium; zusätzliche Funktionen gleichen sie nicht aus.

Bereich	Teilkriterien und maximale Punkte	Total
Fachliche Vollständigkeit, JPA und API	Vereinbarte Fachfunktionen funktionieren (6); JPA-Persistenz und Beziehungen (5); DTOs, Schichtentrennung und nutzbare API (4)	15
Datenmodell, Constraints und Migrationen	Schlüssiges Modell, ER-Diagramm und Datentypen (5); DB-seitige Integrität und begründete Schemaentscheidungen (5); Neuaufbau und nachfolgende Migration (5)	15
Transaktionen und Nebenläufigkeit	Atomare Geschäftsoperation (5); belegter Rollback nach Schreibzugriff (5); wirksamer Schutz vor konkurrierenden Änderungen (5)	15
Query Design, JDBC, View und Pagination	Gezielte JPQL-/DTO-Projektion (3); parametrisierte JDBC-Auswertung mit korrektem JOIN und Aggregation (5); sinnvoll verwendete View (3); DB-seitiger Filter, stabile Sortierung und Pagination (4)	15
Performanceanalyse, Index und ORM-Ladeverhalten	Reproduzierbare grosse Datenmenge (3); vergleichbare Pläne und Messungen mit Interpretation (5); begründeter Index und Optimierung (3); untersuchtes und begründetes ORM-Ladeverhalten (4)	15
Integrationstests und Reproduzierbarkeit	Aussagekräftige automatisierte PostgreSQL-Tests (6); vollständige Start-/Testanleitung und zugeordnete Nachweise (4)	10
Cache und Aktualität	Sinnvoller Einsatz und begründete Strategie (2); Cache-Treffer und Verhalten nach Änderung nachgewiesen (3)	5
Individuelle Verteidigung	Verständliche Demonstration (3); fachliche Begründung einer Entscheidung (4); selbstständige, korrekte Antwort auf Rückfrage (3)	10
Gesamt		100

Bewertungsabstufungen

Jedes Teilkriterium wird einzeln anhand folgender Stufen bewertet. Die erreichten Punkte ergeben sich aus dem Anteil an seinen Maximalpunkten; Viertelpunkte bleiben erhalten.

Stufe	Anteil	Beschreibung
Vollständig erfüllt	100 %	Fachlich korrekt, vollständig und anhand der geforderten Nachweise überprüfbar
Weitgehend erfüllt	75 %	Kern funktioniert und ist belegt; kleinere Fehler oder Lücken
Teilweise erfüllt	50 %	Erkennbarer, teilweise funktionierender Ansatz; wesentliche Aspekte oder Nachweise fehlen
Ansätze vorhanden	25 %	Einzelne brauchbare Teile; überwiegend fehlerhaft oder kaum überprüfbar
Nicht erfüllt	0 %	Fehlt oder kein verwertbarer Nachweis vorhanden

Die technischen Bereiche bewerten die jeweilige Funktion und deren Nachweis. Der Testbereich bewertet zusätzlich die Qualität und Reproduzierbarkeit der automatisierten Tests. Für die Verteidigung gelten dieselben Stufen bezogen auf Demonstration, Begründung und Antwort. Die Notenumrechnung richtet sich nach den kommunizierten Vorgaben des Dozenten.